

Prezentarea 4

Prof. Coordonator: Ș.I.dr.ing.Stelian-Nicolae NICOLA

Mentor: Pătrașcu Hary



Să continuăm de unde am rămas...

În ultimul curs am început să implementăm acțiunile uneltelor.

Primul tool pe care l-am implementat este lopata. Dar în acest moment, terenul nu are autotiling și nu arată foarte bine. Așa că, haideți să implementăm autotiling pentru amplasarea tile-urilor de dirt de data aceasta.

Singurele schimbări pe care trebuie să le facem este să apelăm metoda 'set_cells_terrain_connect' în loc de 'set_cell' pe layer-ul de dirt.

Metoda set_cells_terrain_connect primește 4 argumente, un array de poziții, id-ul terrain set-ului (posibil să fie 0 la toată lumea), id-ul terenului din terrain set și ignore_empty_terrains.

În primul rând, să modificăm ce metodă apelăm pe dirt layer:

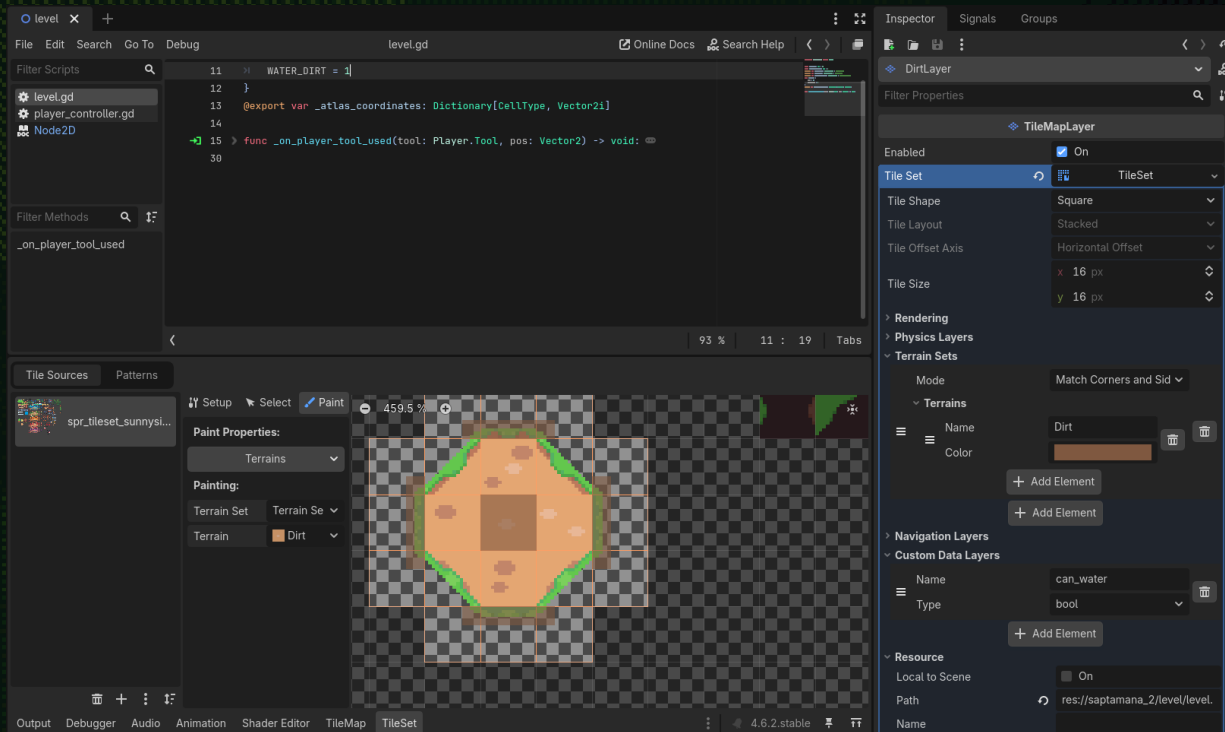
```
→ 15 ▼ func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:  
16   >| var grid_pos: Vector2i = Vector2i(int(pos.x / TILE_SIZE), int(pos.y / TILE_SIZE))  
17   ▼ >| match tool:  
18   ▼ >| >| Player.Tool.SHOVEL:  
19   >| >| >| # dirt_layer.set_cell(grid_pos, 0, _atlas_coordinates[CellType.DIRT])  
20   >| >| >| dirt_layer.set_cells_terrain_connect([grid_pos], 0, 0, true)
```



Următorul pas ar fi să creăm autotiling pe dirt layer.

În primul rând, selectați nodul DirtLayer din node tree, deschideți tileset-ul nodului(click pe el) și mergeți la tileset. În tabul de inspector, la terrain sets creați un terrain set nou și adăugați un element cu numele Dirt. În tabul TileSet, mergeți la paint și selectați terrains, apoi selectați terrain set-ul creat și terenul Dirt pe care l-ați creat.

Așa ar trebui să fie pictat terenul...



Dacă rulați jocul și selectați lopata, o să observați că nu se va amplasa niciun tile de teren, acest lucru se datorează faptului că proprietatea de `ignore_empty_terrains` este setată pe `true` când apelăm metoda pe `dirt_layer`.

```
func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:
>|   var grid_pos: Vector2i = Vector2i(int(pos.x / TILE_SIZE), int(pos.y / TILE_SIZE))
>|   match tool:
>|   >|   Player.Tool.SHOVEL:
>|   >|   >|   # dirt_layer.set_cell(grid_pos, 0, _atlas_coordinates[CellType.DIRT])
>|   >|   >|   dirt_layer.set_cells_terrain_connect([grid_pos], 0, 0, false)
>|   >|   Player.Tool.WATER:
```



Dacă ați urmat pașii în mod corect, o să vedeți când rulați jocul și testați că terenurile de dirt se vor interconecta când folosiți lopata.

Acum că am implementat lopata, pasul logic următor ar fi să implementăm o modalitate de a uda pământul. Să gândim algoritmul pas cu pas.

Cum am zis și în prezentarea trecută, prima etapă de a rezolva o problemă este să punem întrebările corecte.

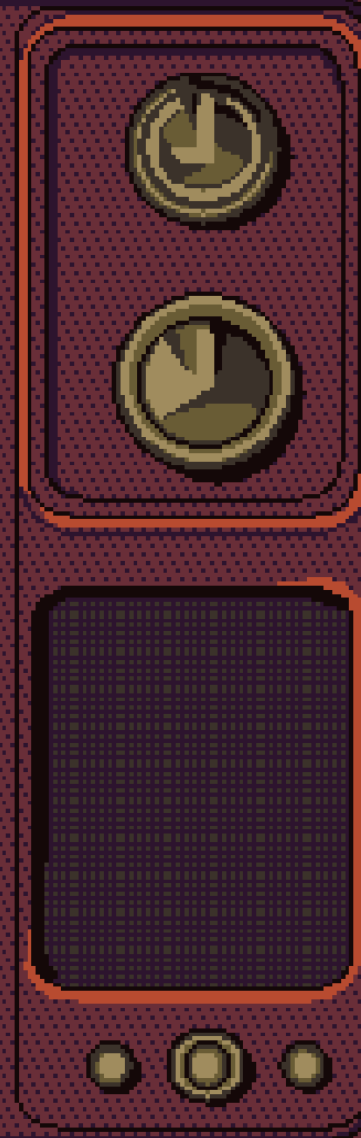
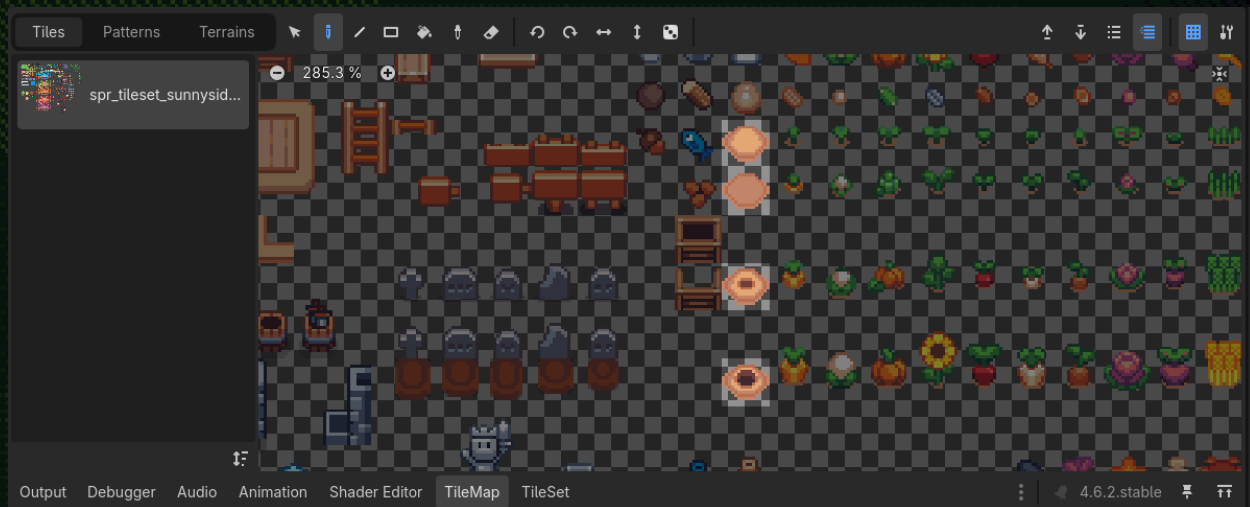
Exercițiu: Încercați să scrieți un pseudocod/ logică în cuvinte (folosind comenturi # sau pe foaie) pentru cum ar trebui să implementăm udarea pământului.



În primul rând să creem un tilemap layer nou, de data asta să conțină tile-uri cu textura udată.

Exercițiu: Atașați un nod tilemap layer în ierarhia de noduri și numiți-l WaterDirtLayer.

Adăugați un tileset la acest nod. Apoi selectați următoarele tile-uri:



Acum că avem un nou tilemap layer, putem să revenim la pseudocod.

```
21 ❏ ❏ ❏ # 1. dacă suntem în cazul tool == water
22 ❏ ❏ ❏ # 2. verific că tile-ul pe care încerc să îl ud este de tip dirt
23 ❏ ❏ ❏ # -> adică verific pe dirt layer
24 ❏ ❏ ❏ # -> adică verific că poziția pe care încerc să o ud are un dirt tile
25 ❏ ❏ ❏ # -> cum verific că există un tile la o poziție în Godot?
26 ❏ ❏ ❏ # 3. verific că este un tile pe care putem să îl udăm(nu a fost udat deja
27 ❏ ❏ ❏ # sau nu este tile de margine)
28 ❏ ❏ ❏ # -> adică am nevoie de un algoritm SAU un fel de proprietate atașată pe anumite
29 ❏ ❏ ❏ # tile-uri
30 ❏ ❏ ❏ # -> cum atașez proprietăți custom la tile-uri în Godot?
31 ❏ ❏ ❏ # 4. dacă este un tile udabil, amplasez un tile de apă
```



Recomand această tehnică de a scrie cod când sunteți începători sau chiar și când aveți multă experiență și încercați să implementați anumiți algoritmi complecși. Chiar dacă pseudocod-ul nu este perfect, o să puteți să reiterați pe el până când iese implementarea pe care v-ați propus-o. Un alt mod de a rezolva o problemă este de a vorbi cu altcineva despre ce încercați să implementați(chiar și singuri, cu vocea voastră), dacă nimic nu merge, cel mai sigur este să luați o pauză și soluția/ alte variante vor veni natural când nu vă așteptați.



Exercițiu: cu pașii pe care i-ați scris în pseudocod, încercați să implementați codul propriu-zis acum.

```
>| # 1. dacă suntem în cazul tool == water
>| Player.Tool.WATER:
>| >| # 2. verifică că tile-ul pe care încerc să îl ud este de tip dirt
>| >| # -> adică verifică pe dirt layer
>| >| # -> adică verifică că poziția pe care încerc să o ud are un dirt tile
>| >| # -> cum verifică că există un tile la o poziție în Godot? -> WELCOME TO THE INTERNET
>| >| var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
>| >| # altfel(dacă nu e tile de dirt)] încheie execuția
>| >| if not dirt_data:
>| >| >| return
```




Pentru pasul 3, formulați o întrebare și căutați pe Internet cum să atașați date pe anumite tile-uri în Godot.

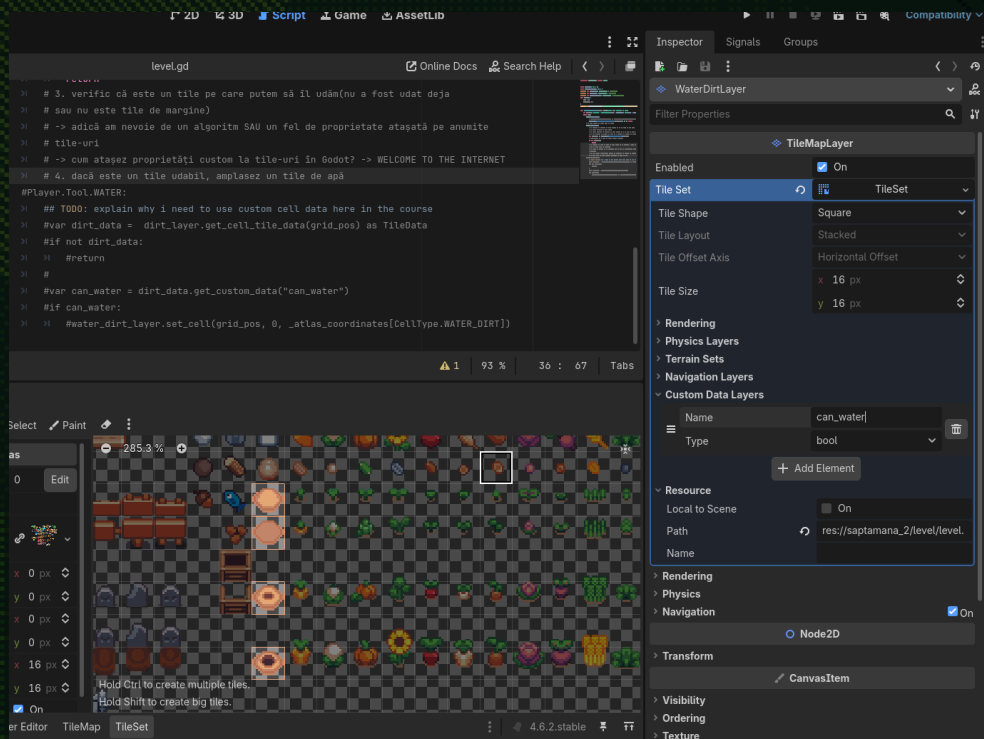
Custom data layers

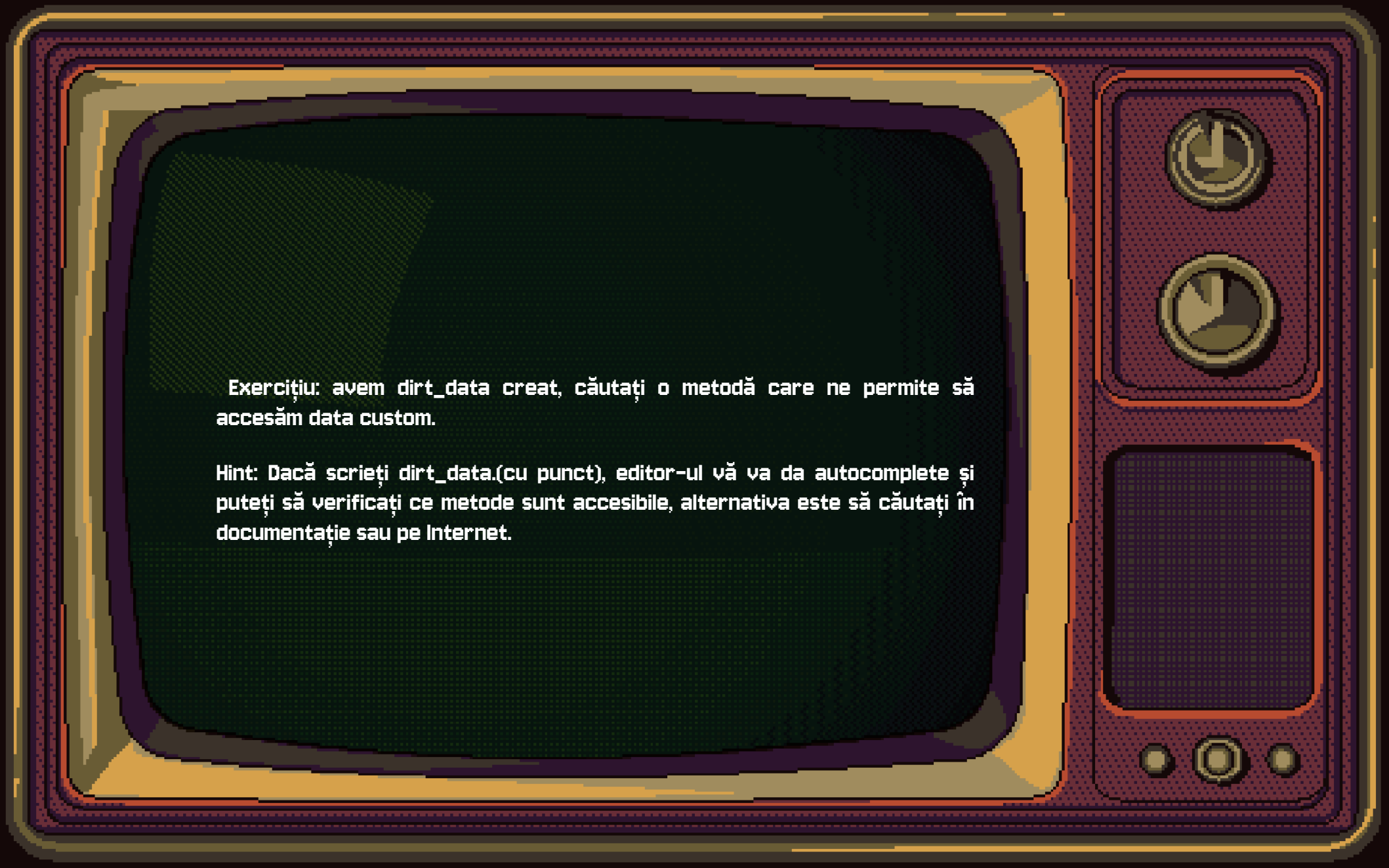
Custom data layers sunt modalitate din Godot de a adăuga date custom la anumite tile-uri, de exemplu poate vrem să facem un tile care poate să ia foc, dar altele nu, sau un tile pe care jucătorul să alunece, etc.

Cum le folosim?

Selectăm nodul WaterDirtLayer creat anterior și proprietatea tileset din inspector, în același loc unde ați creat terrain sets, puteți să creați și date custom dând click pe custom data layers. Dați nume datei create "can_water" sau orice vreți voi doar să tineți minte, puteți să îi dați tipul boolean(bool).







Exercițiu: avem `dirt_data` creat, căutați o metodă care ne permite să accesăm data custom.

Hint: Dacă scrieți `dirt_data.(cu punct)`, editor-ul vă va da autocomplete și puteți să verificați ce metode sunt accesibile, alternativa este să căutați în documentație sau pe Internet.



```
# 3. verific că este un tile pe care putem să îl udăm(nu a fost udat deja  
# sau nu este tile de margine)  
# -> adică am nevoie de un algoritm SAU un fel de proprietate atașată pe anumite  
# tile-uri  
# -> cum atașez proprietăți custom la tile-uri în Godot? -> WELCOME TO THE INTERNET  
var can_water = dirt_data.get_custom_data("can_water")  
# 4. dacă este un tile udabil, amplasez un tile de apă|
```



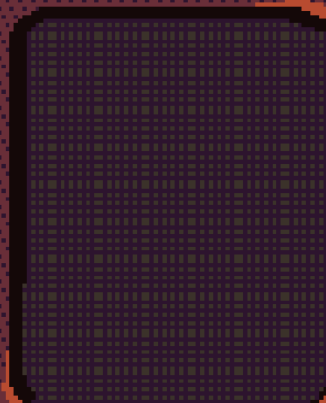
Exercițiu: la pasul 4, încercați să faceți voi implementarea, știm că avem water dirt layer și o metodă de set cell...

```
# -> adică am nevoie de un algoritm SAU un fel de proprietate atașată pe anumite  
# tile-uri  
# -> cum atașez proprietăți custom la tile-uri în Godot? -> WELCOME TO THE INTERNET  
var can_water = dirt_data.get_custom_data("can_water")  
# 4. dacă este un tile udabil, amplasez un tile de apă  
if can_water:|  
>|  water_dirt_layer.set_cell(grid_pos, 0, _atlas_coordinates[CellType.WATER_DIRT])
```

Observăm că am ratat un pas în pseudocod, anume pasul de a stoca coordonatele pentru tile-ul de dirt udat. Ratarea pașilor se va întâmpla în mod natural dar se va rezolva de la sine când veți începe implementarea codului propriu-zisă.

Exercițiu: Adăugați un cell type nou, water dirt și în atlas coordinates un nou key-value pair cu cheia water dirt și coordonatele pe care le găsiți în tileset.

Pentru a adăuga valori noi în atlas coordinates, selectați nodul level unde este atașat scriptul, și în inspector o să vedeți atlas coordinates deoarece este o variabilă de tip export.



Filter Properties

☒ LevelManager

Atlas Coordinates Dictionary[int, Vector2i] (size 2)

Dirt	▼	x 6	◇	🗑
		y 35	◇	
Water Dirt	▼	x 50	◇	🗑
		y 12	◇	
New Key:		Dirt	▼	
New Value:		x 0	◇	
		y 0	◇	

+ Add Key/Value Pair

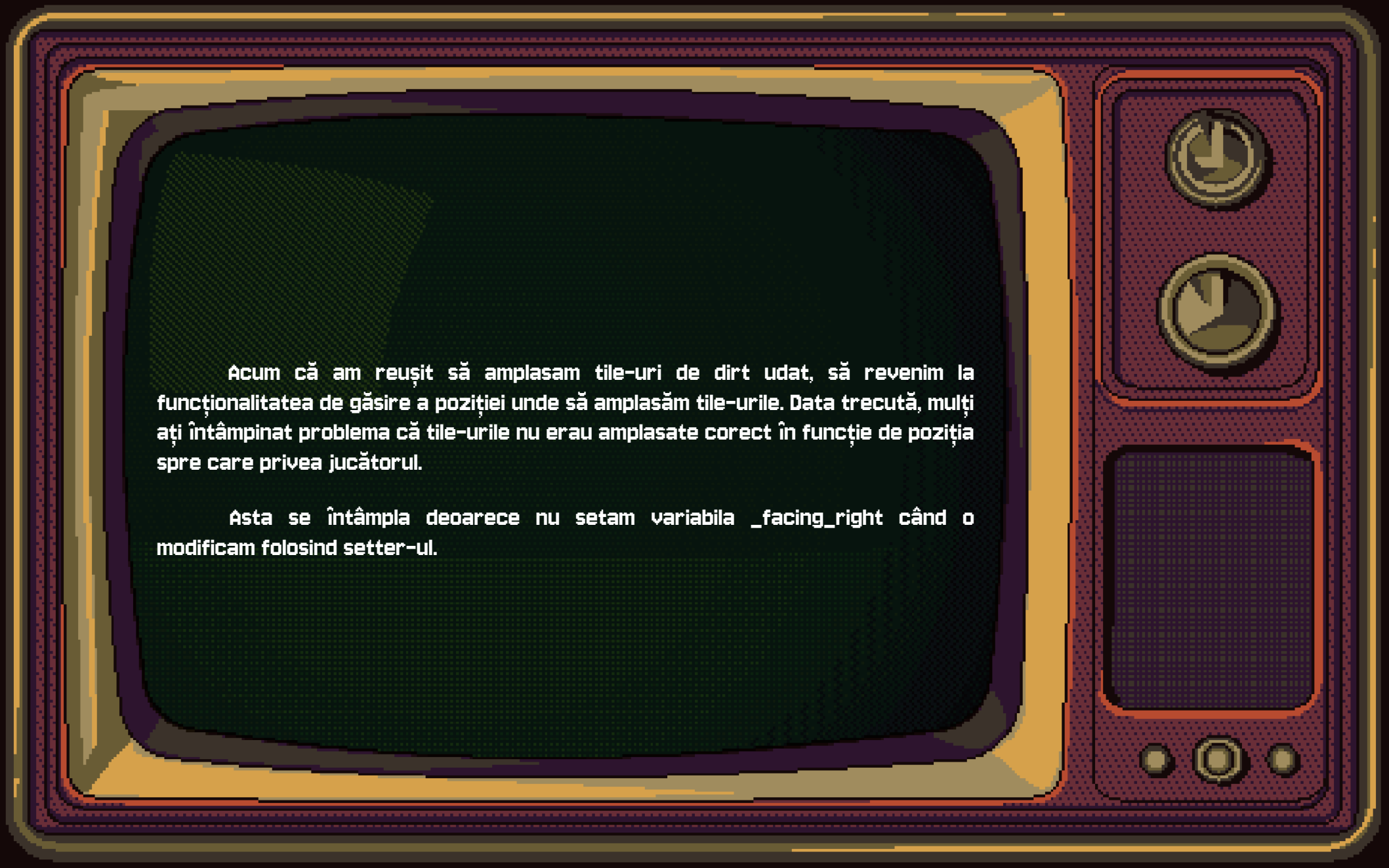
☒ Node2D

▼ Transform



Când rulați jocul, dacă creați tile-uri de pământ cu lopata, le veți putea uda apoi.



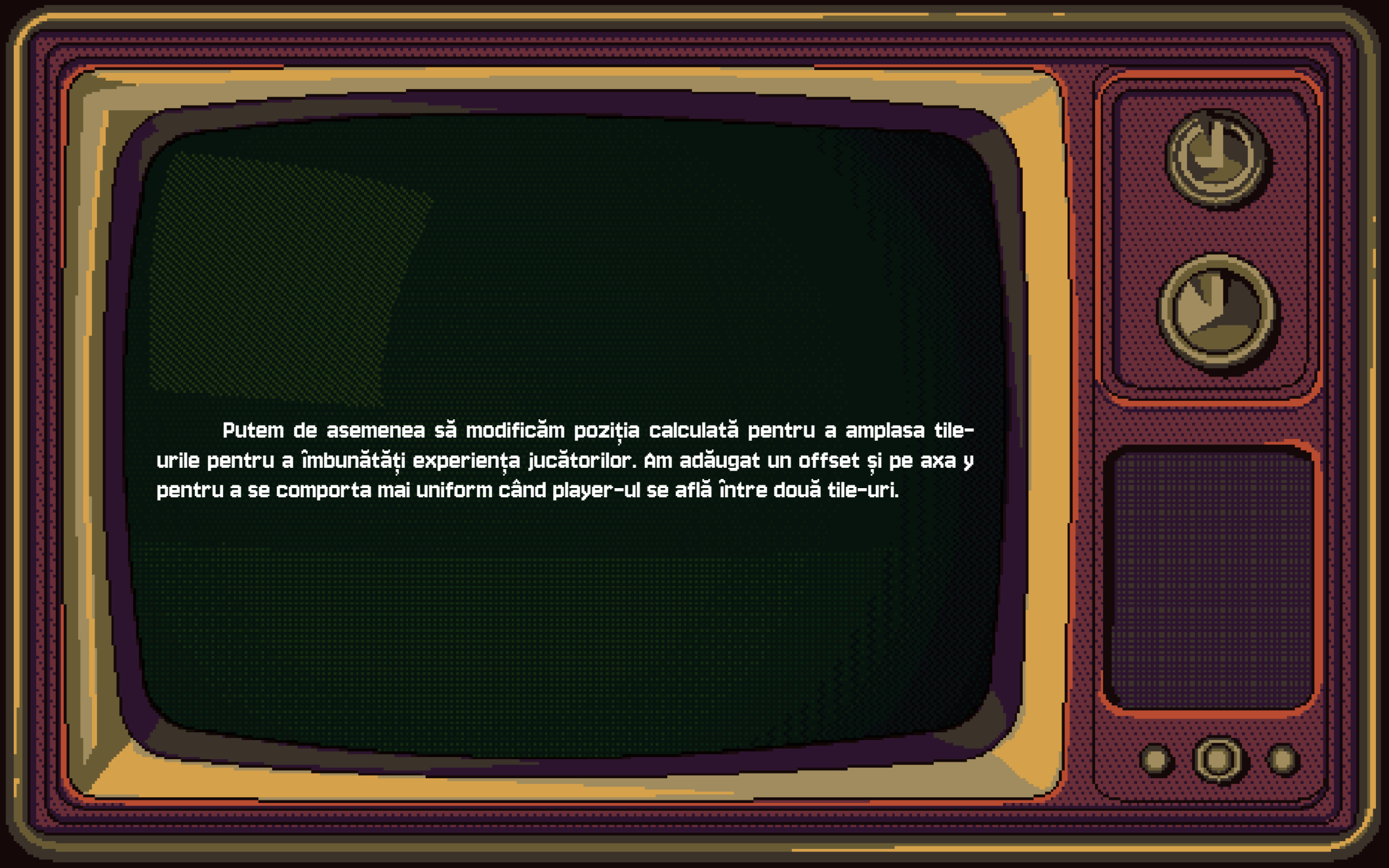


Acum că am reușit să amplasăm tile-uri de dirt udat, să revenim la funcționalitatea de găsim a poziției unde să amplasăm tile-urile. Data trecută, mulți ați întâmpinat problema că tile-urile nu erau amplasate corect în funcție de poziția spre care privea jucătorul.

Asta se întâmpla deoarece nu setam variabila `_facing_right` când o modificam folosind setter-ul.



```
22  ▼ var _facing_right: bool = false:
23  ▼ |  set(value):
24    |  |  _facing_right = value|
25    |  |  character_animated_sprite.flip_h = not _facing_right
26    |  |  tools_animated_sprite.flip_h = not _facing_right
```



Putem de asemenea să modificăm poziția calculată pentru a amplasa tile-urile pentru a îmbunătăți experiența jucătorilor. Am adăugat un offset și pe axa y pentru a se comporta mai uniform când player-ul se află între două tile-uri.

În scriptul player.gd, la pos_offset, am adăugat un offset de 6 unități pe axa y

```
▼ func _play_tool_animation() -> void:
    >I # error handling demonstration
    >I var _tool_anim: String = _tool_anims.get(_current_tool, "")
    >I character_animated_sprite.play(_tool_anim)
    >I tools_animated_sprite.play(_tool_anim)
    >I
    >I # wait for animation to finish to reset stated
    >I await character_animated_sprite.animation_finished
    >I await tools_animated_sprite.animation_finished
    >I
    >I # apply tool effects only after the animation has finished
    >I var tool_dir: float = TOOL_RIGHT if _facing_right else TOOL_LEFT
    >I var pos_offset: Vector2 = position + Vector2(tool_dir * TOOL_OFFSET, 6.0)
    >I tool_used.emit(_current_tool, pos_offset)
    >I # go back to the initial state
    >I _set_state(State.IDLE)
```

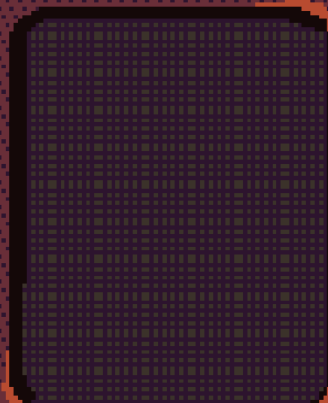
De asemenea, putem modifica calcularea poziției de pe grid folosind funcția matematică `floor` în loc de castarea la `int`. `floor` oferă un comportament uniform când ne aflăm în coordonate negative `-x`, `-y`. Există și alte metode precum folosirea metodei `local_to_map()` pentru a nu calcula ‘de mână’ aceste coordonate.

```
15 ▼ func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:
16   >| var grid_pos: Vector2i = Vector2i(floor(pos.x / TILE_SIZE), floor(pos.y / TILE_SIZE))
17   >| print(grid_pos)
18   >| print(pos)
```


Răspuns la întrebarea unui coleg legat de proprietatea `fliph`. În implementarea funcționalității de calculare a poziției pentru tool, v-am spus că puteți folosi o metodă matematică sau o metodă vizuală folosind noduri.

Dacă folosiți proprietatea `fliph`, engine-ul va da flip la sprite, ci nu la nod, astfel chiar dacă nodul `Marker2D` este un copil al sprite-ului, acesta nu își va modifica poziția.

Pentru a da flip la marker împreună cu sprite-ul, trebuie să modificați proprietatea de scale din `1.0` în `-1.0` și invers și marker-ul să fie un nod copil al nodului de sprite. Sau există modalitatea să calculați simetricul din cod :^)



The image shows a pixelated, retro-style interface. On the left is a large, dark, rounded rectangular area that serves as a screen. On the right is a control panel with a textured, reddish-brown background. At the top of the panel are two circular buttons with a metallic, 3D appearance. Below these is a rectangular area with a fine grid pattern. At the bottom of the panel are three small circular buttons, the middle one being slightly larger than the two on the sides. The entire interface is framed by a thick, multi-layered border in shades of yellow, orange, and red.

Să trecem la implementarea următoarei funcționalități. Acum că putem săpa pământul și să îl udăm, să implementăm un sistem de plantare.

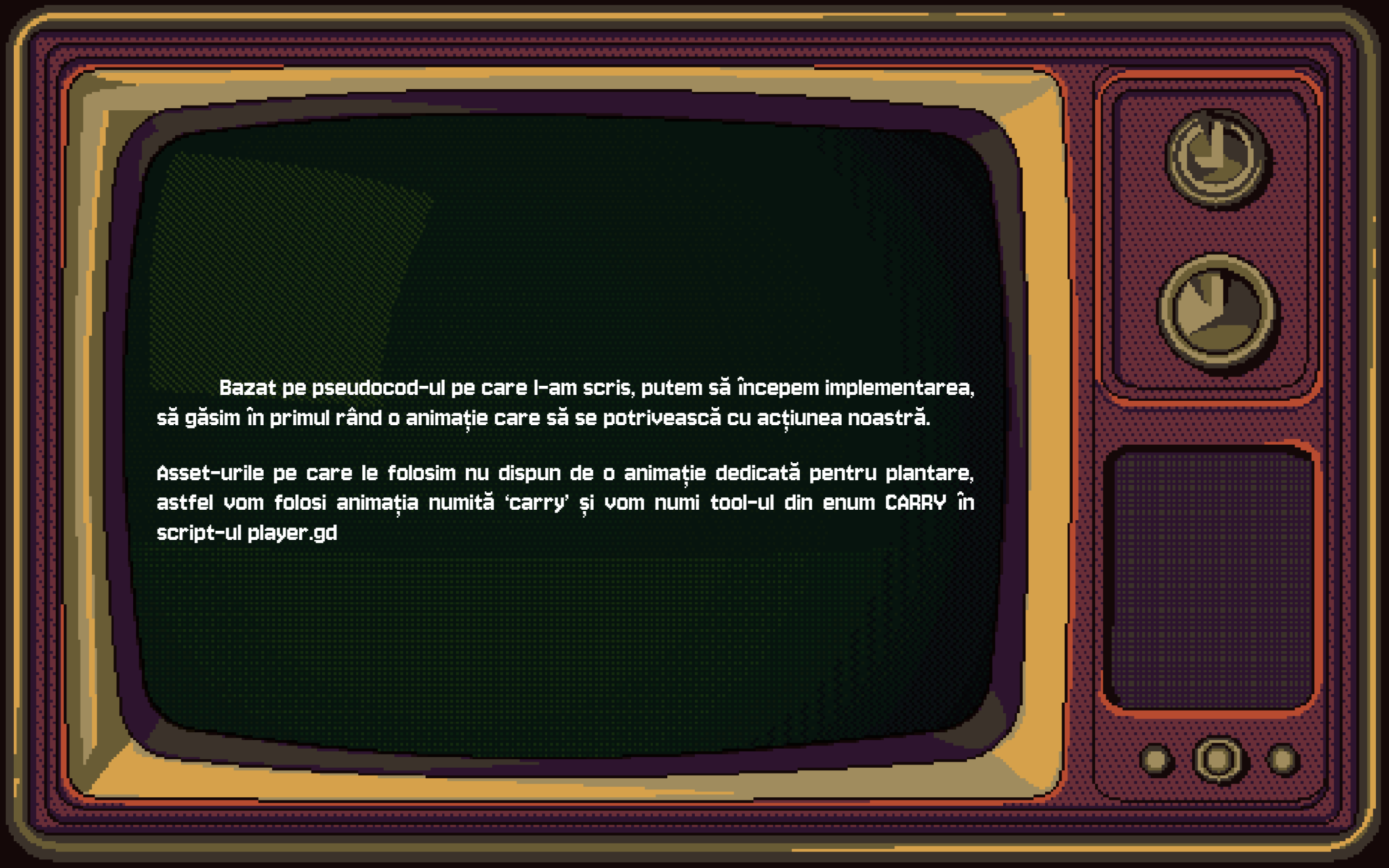
Exercițiu: scrieți folosind comment-uri în cod cum ați implementa acest feature.



```
4 1. Dacă actorul apasă click
5 atunci trimit semnalul de poziție a unelei la level
6 2. dacă unealta este o unealtă de plantat
7 atunci verific dacă tile-ul este tile săpat(dirt layer)
8 dacă e tile săpat, creez o plantă la acea poziție
9 altfel închei|
10
```

The image shows a pixel-art style handheld device, possibly a PDA or a handheld console. It has a large screen on the left and a control panel on the right. The control panel includes two circular buttons with directional arrows, a rectangular button with a grid pattern, and three small circular buttons at the bottom. The screen displays a text box with a light gray background.

2. dacă unealta este o unealtă de plantat (-> implică că am nevoie de o animație pentru acea uneltă), (-> implică și că am nevoie de o variabilă în enumul de Tool-uri pentru a avea cu ce să comparăm)
atunci verific dacă tile-ul este tile săpat(dirt layer)
dacă e tile săpat, creez o plantă (-> implică că am nevoie de o scenă /obiect care să reprezinte acea plantă) la acea poziție|
altfel închei

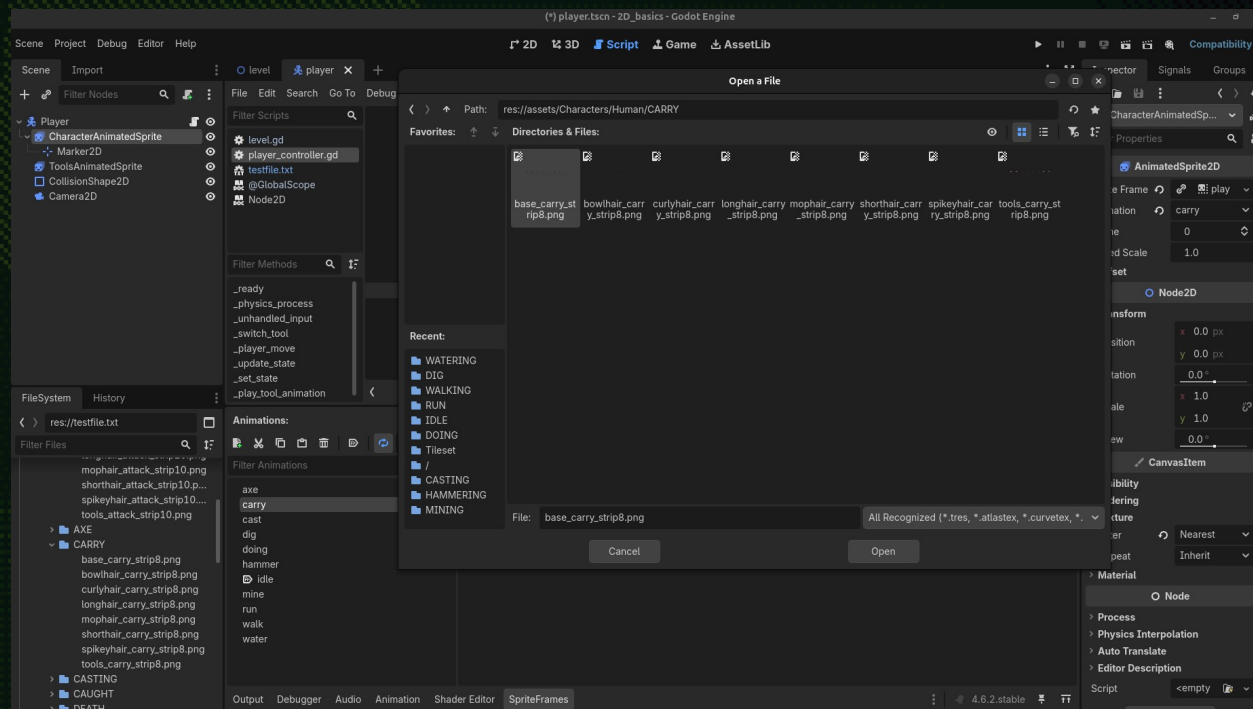


Bazat pe pseudocod-ul pe care l-am scris, putem să începem implementarea, să găsim în primul rând o animație care să se potrivească cu acțiunea noastră.

Asset-urile pe care le folosim nu dispun de o animație dedicată pentru plantare, astfel vom folosi animația numită 'carry' și vom numi tool-ul din enum CARRY în script-ul `player.gd`

```
28  enum Tool {  
29      >|  AXE,  
30      >|  SHOVEL,  
31      >|  CARRY,|
```

Adăugați animația carry pentru ambele animated sprites





De fiecare dată când facem o modificare este bine să rulăm programul pentru a verifica că totul funcționează cum trebuie până la acel punct, deci rulați programul și spuneți dacă întâmpinați probleme.

The image depicts a pixelated, retro-style interface, possibly a terminal or a control panel. It features a large, dark, rounded rectangular screen on the left side. To the right of the screen is a control panel with a textured, reddish-brown background. This panel includes two circular buttons at the top, a rectangular area with a grid pattern in the middle, and three small circular buttons at the bottom. The entire interface is framed by a thick, multi-layered border in shades of yellow, orange, and brown.

Să trecem la script-ul level.

Exercițiu: gândiți și implementați cazul când folosim noul tool, CARRY. Puteți să vă inspirați din codul anterior scris și să faceți un pseudocod în comment-uri, de asemenea puneți un print acolo unde ar trebui să 'spawnăți' planta.

```
42  ▾ ▶ ▶ ▶ Player.Tool.CARRY:
43  ▶ ▶ ▶ ▶ # daca tile-ul pe care incerc sa il vd este de tip dirt
44  ▶ ▶ ▶ ▶ var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
45  ▶ ▶ ▶ ▶ # altfel inchei
46  ▾ ▶ ▶ ▶ ▶ if not dirt_data:
47  ▶ ▶ ▶ ▶ ▶ return
48  ▶ ▶ ▶ ▶ print("am spawnat o planta!")
49
```



Acum că avem codul relativ complet, putem să creăm scena plantei. Dar, în pseudocod nu am creat pașii pentru această etapă, așa că putem să reiterăm pe pseudocod să includă pașii pentru a crea scena plantelor(de exemplu de ce noduri ar avea nevoie acea scenă).



```
12 altfel închei
13
14 1. creez o nouă scenă, nodul de bază node 2d
15 2. am nevoie de o modalitate de a afișa o plantă(-> implică că am nevoie
16 de un nod vizual sprite2d) dar știu că am mai multe poze pentru fiecare tip diferit
17 de plantă așadar folosesc un nod animatedsprite2d.
18 3. adaug animațiile pentru tipurile diferite de plante
19 4. plantele ar trebui să aibă și un collision box pentru ca player-ul să nu poată trece prin ele
20 |
```



Exercițiu: creați un folder nou numit plant. În acel folder creați o scenă cu nodul de bază Node2D(2D scene), numiți scena plant. Adăugați scenei plant un animated sprite 2D și creați un sprite frames resource din tabul inspector. În folder-ul assets se află un folder crops la /elements/crops, adăugați o animație nouă numită carrot și folosiți imaginile carrot_00, carrot_01, etc.

Acum că avem scena creată, în loc să printăm un mesaj putem să amplasăm direct planta la locul potrivit. Pentru asta, avem nevoie de o referință la scena plantă în script-ul level.gd. Din sistemul de fișiere puteți da CLICK apoi CTRL + drag and drop în script la scena plant.tscn și engine-ul va crea automat o variabilă care reprezintă scena voastră.

```
13  @export var _atlas_coordinates: Dictionary[CellType, Vector2i]
14  const PLANT = preload("uid://b7khh4l3xx15g")
15
```


preload

Preload este o metodă din GDScript care încarcă o referință la o scenă în program. Această primește un path de unde poate găsi acea resursă în proiect sau un uid(un id specific fiecărei resurse autogenerat de engine, practic ca un CNP).

instantiate

Cu această resursă încărcată în program putem să apelăm metoda `instantiate()` pentru a 'spawna' acea scenă și copii ei.

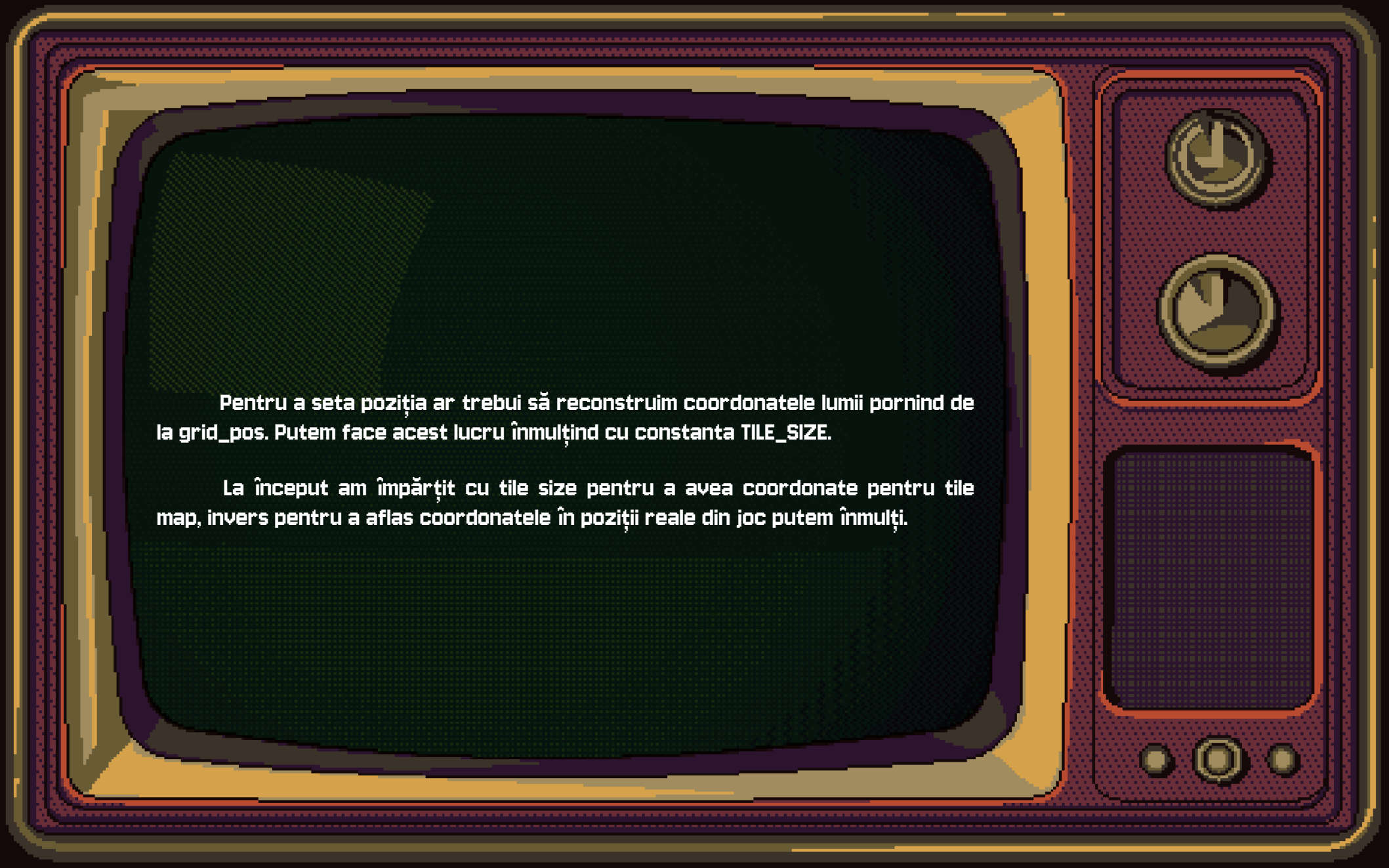
add_child

Ultima etapă este să adăugăm acel nod spawnat ca și copil unui alt nod. Dacă apelăm metoda `add_child(nod_creat)` sau `nume_nod.add_child(nod_creat)`, scena va fi adăugată în arborele de noduri sub un părinte. De exemplu dacă apelăm `add_child(nume_nod)` simplu, scena va fi adăugată ca și copilul nodului `level` deoarece scriptul din care am apelat metoda este atașat nodului `level`.

```
| Player.Tool.CARRY:
| >| # daca tile-ul pe care incerc sa il ud este de tip dirt
| >| var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
| >| # altfel inchei
| >| if not dirt_data:
| >| >| return
| >| print("am spawnat o planta!")
| >| var plant = PLANT.instantiate()
| >| add_child(plant)|
```



Dacă rulăm codul acum și încercăm să plantăm ceva, vom observa că planta nu se află la poziția care trebuie. Asta deoarece nu am schimbat poziția nodului când l-am spawnat.



Pentru a seta poziția ar trebui să reconstruim coordonatele lumii pornind de la `grid_pos`. Putem face acest lucru înmulțind cu constanta `TILE_SIZE`.

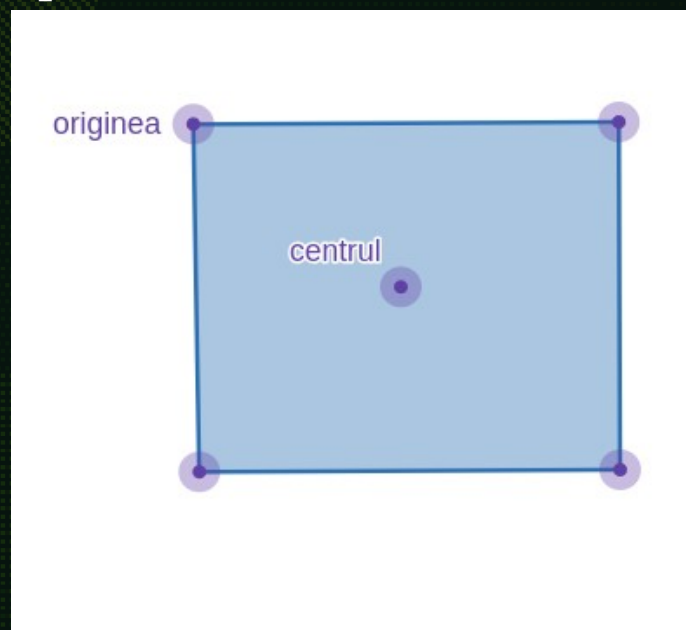
La început am împărțit cu `tile size` pentru a avea coordonate pentru `tile map`, invers pentru a afla coordonatele în poziții reale din joc putem înmulți.

Dacă rulăm jocul observăm că plantele nu sunt amplasate în centrul tile-ului de pământ, pentru asta ar trebui să adăugăm un offset din nou.

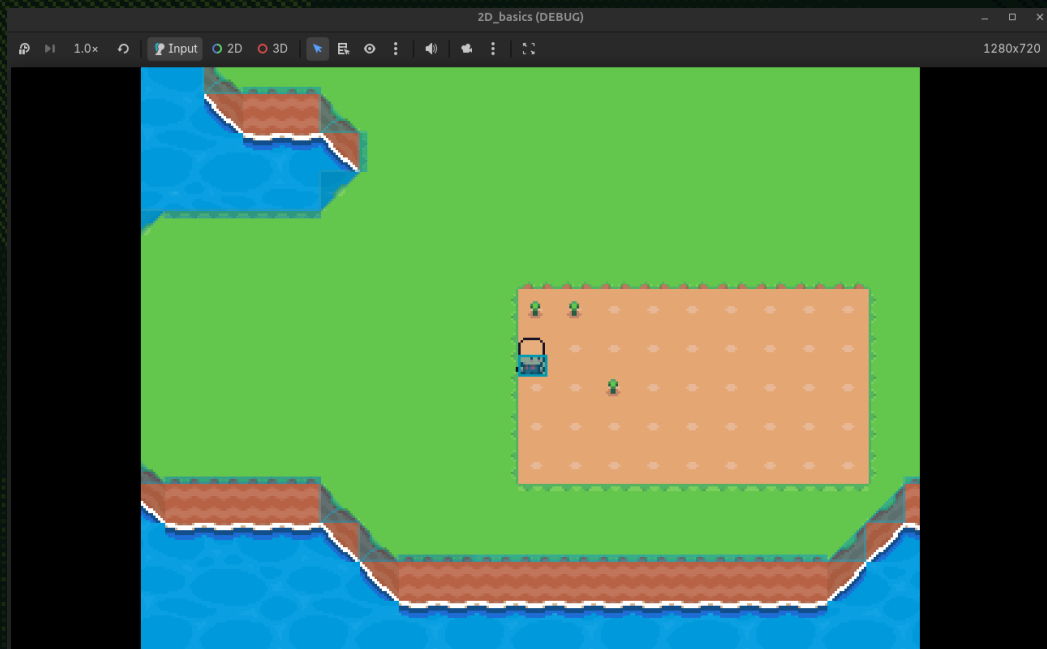
```
>| var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
>| # altfel inchei
>| if not dirt_data:
>|   >| return
>| print("am spawnat o planta!")
>| var plant = PLANT.instantiate()
>| print(grid_pos * TILE_SIZE)
>| plant.position = (grid_pos * TILE_SIZE)
>| add_child(plant)
```

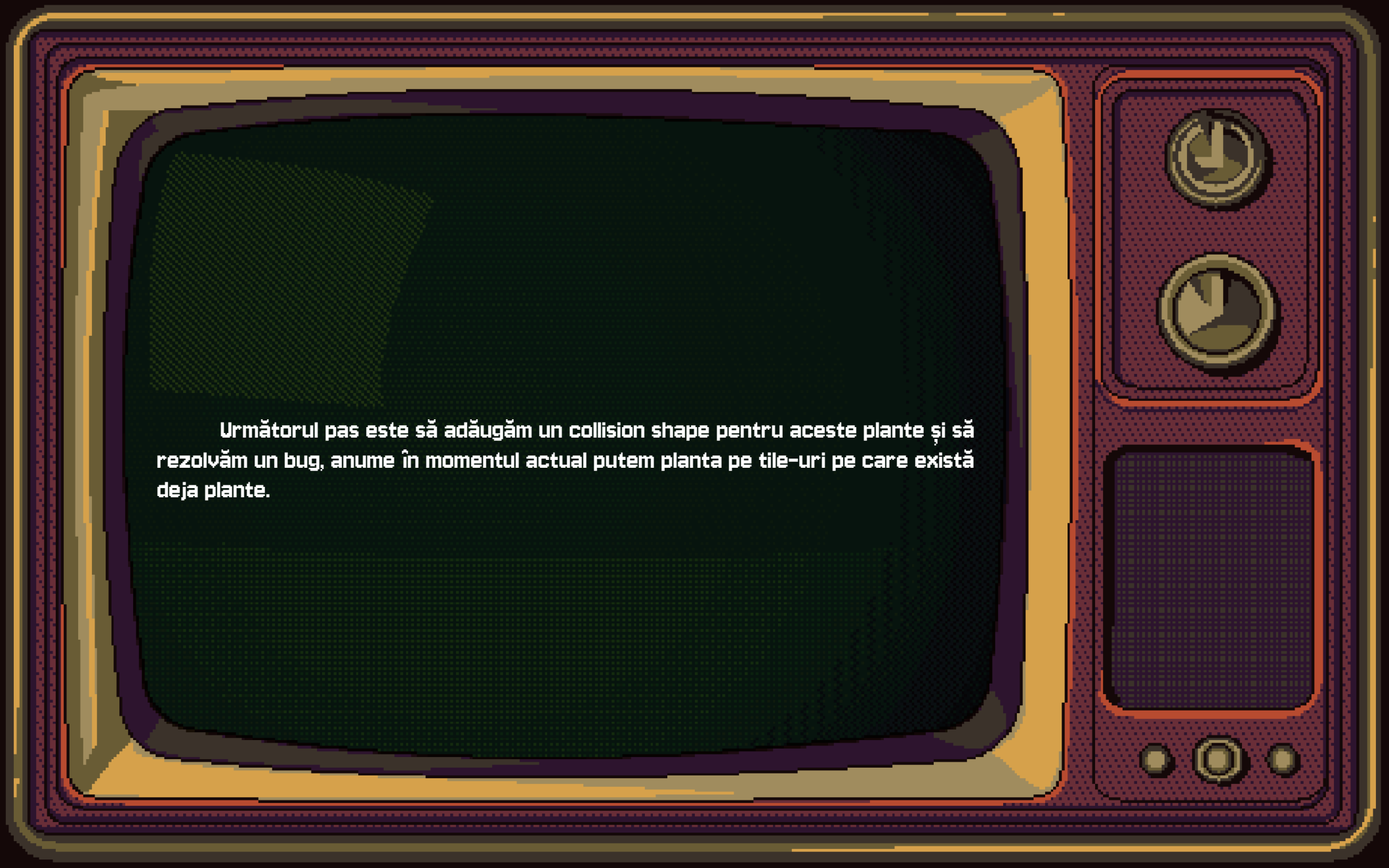



Să zicem că tile-ul nostru este 2x2 ca dimensiune, dacă vrem centrul trebuie relativ la origine trebuie doar să împărțim la 2 mărimea tile-ului rezultă 1x1. Ce mai rămâne este să adunăm acest rezultat la originea noastră.

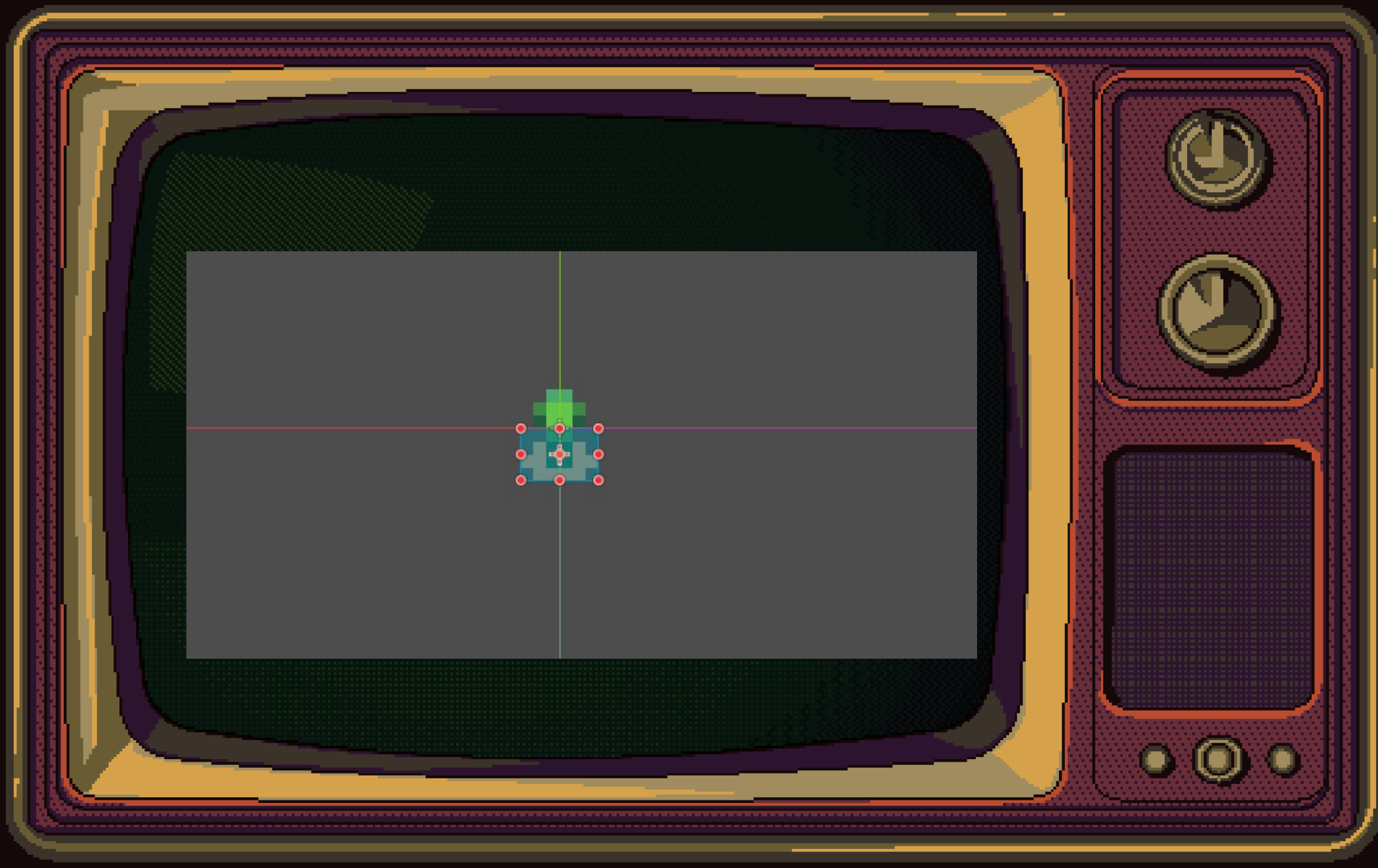


```
>| Player.Tool.CARRY:
>| >| # daca tile-ul pe care incerc sa il ud este de tip dirt
>| >| var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
>| >| # altfel inchei
>| >| if not dirt_data:
>| >| >| return
>| >| print("am spawnat o planta!")
>| >| var plant = PLANT.instantiate()
>| >| print(grid_pos * TILE_SIZE)
>| >| plant.position = (grid_pos * TILE_SIZE) + Vector2i(TILE_SIZE / 2, TILE_SIZE / 2)
>| >| add_child(plant)
```





Următorul pas este să adăugăm un collision shape pentru aceste plante și să rezolvăm un bug, anume în momentul actual putem planta pe tile-uri pe care există deja plante.





Observăm că după ce adăugăm collision shape-ul, engine-ul ne va avertiza că nu va funcționa, asta deoarece nodul părinte nu este un nod care are fizică.

Exercițiu: Schimbați tipul nodului părinte Plant de la Node2D la StaticBody2D



Exercițiu: gândiți-vă la o modalitate de a rezolva bug-ul care permite
plantarea pe același tile de mai multe ori.

The image shows a pixel-art style interface. On the left is a large, dark, rounded rectangular screen. On the right is a control panel with a textured background. It features two circular gauges at the top, a rectangular display area in the middle, and three small circular buttons at the bottom. The entire interface is framed by a thick, multi-layered border.

În primul rând, ar trebui să memorăm tile-urile plantate într-un array de fiecare dată când amplasăm o plantă. Înainte să plantăm, verificăm dacă poziția de grid nu se află deja în array-ul de poziții.

Exercițiu: 1. Creați un array care stochează Vector2(poziții)
2. verificați că poziția de grid nu se află în array
3. adăugați poziția în array și plantați planta



```
16  var planted_cells: Array[Vector2i]  
17
```

```
45  ▾ | | | Player.Tool.CARRY:
46  | | | | # daca tile-ul pe care incerc sa il vd este de tip dirt
47  | | | | var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
48  | | | | # altfel inchei
49  ▾ | | | | if not dirt_data:
50  | | | | | return
51  | | | |
52  ▾ | | | | if not grid_pos in planted_cells:
53  | | | | | var plant = PLANT.instantiate()
54  | | | | | add_child(plant)
55  | | | | | position = (grid_pos * TILE_SIZE) + Vector2i(TILE_SIZE / 2, TILE_SIZE / 2)
56  | | | | | planted_cells.append(grid_pos)
57
```

Ultimul pas este să adăugăm un nod care să conțină toate aceste plante. Deoarece plantele se spawnează sub player, acestea vor fi afișate deasupra player-ului.

Exercițiu: Adăugați un node2d simplu numit Plants deasupra player-ului în scena level.

În script-ul level luați o referință la acel nod creat și când plantați, apălați `Plants.add_child(plant)` în loc de `add_child()`, acest apel va adăuga plantele ca noduri copil sub nodul Plants în loc de copii ai nodului Level.

```
13  @export var _atlas_coordinates: Dictionary[CellType, Vector2i]
14  const PLANT = preload("uid://b7kxb4l3xx15g")
15  @onready var plants: Node2D = $Plants
16
17  ▸ func _on_player_tool_used(tool: Player.Tool, pos: Vector2) -> void:
18  ▸   var grid_pos: Vector2i = Vector2i(floor(pos.x / TILE_SIZE), floor(pos.y / TILE_SIZE))
19  ▸   print(grid_pos)
```

```
45  ▾ | | | Player.Tool.CARRY:
46  | | | | # daca tile-ul pe care incerc sa il vd este de tip dirt
47  | | | | var dirt_data := dirt_layer.get_cell_tile_data(grid_pos)
48  | | | | # altfel inchei
49  ▾ | | | | if not dirt_data:|
50  | | | | | return
51  | | | |
52  ▾ | | | | if not grid_pos in planted_cells:
53  | | | | | var plant = PLANT.instantiate()
54  | | | | | plants.add_child(plant)
55  | | | | | plant.position = (grid_pos * TILE_SIZE) + Vector2i(TILE_SIZE / 2, TILE_SIZE / 2)
56  | | | | | planted_cells.append(grid_pos)
57
```